Module 10 — Monitoring, Observability & Tracing

> **Goal:** Instrument an LLM system end-to-end so you can answer, within minutes: *Is it up? Is it fast? Is it correct? Is it safe? What does it cost, and who is spending it?* Covers OpenTelemetry GenAI semantic conventions, LangFuse, Arize Phoenix/OpenInference, serving metrics (TTFT/ITL/TPS), cost attribution, hallucination-drift detection and SLOs.

Table of Contents

1. Module Overview & First-Principles Architecture
2. Mathematical Foundations
3. Production-Ready Code
4. Hard-Learned Production Failures
5. Interview Questions

---

1. Module Overview & First-Principles Architecture

1.1 The four pillars for LLM systems

Classic observability = metrics, logs, traces. LLM systems add a fourth: **evaluations** (quality signals computed on content).

Pillar	Question	LLM-specific content
Metrics	Is it healthy at aggregate level?	TTFT, ITL, TPS, queue depth, KV-cache %, token counts, cost, refusal rate, guardrail block rate, cache hit rate
Traces	What happened in *this* request?	Span tree: gateway → guardrails → cache → retrieval → rerank → LLM call(s) → tools → output rails; prompts, completions, model/version, token usage
Logs	What exactly was said/happened?	Structured events; sampled prompt/response payloads (PII-redacted)
Evals	Is the content good/safe?	Online judge scores (groundedness, relevance), user feedback, hallucination flags

1.2 Reference observability architecture

```
flowchart LR
  subgraph App["Application / agent runtime"]
    A1[FastAPI gateway]-->A2[Guardrails]-->A3[Retriever]-->A4[LLM client]-->A5[Tools]
  end
  subgraph Serve["Inference"]
```

```
    V[vLLM /metrics + OTLP traces]
    G[DCGM exporter GPU]
  end
  App -- OTLP/gRPC --> C[OTel Collector<br/>batch · redact · tail-sample · route]
  V -- OTLP --> C
  C --> LF[LangFuse<br/>LLM traces, prompts, scores, cost]
  C --> PH[Arize Phoenix<br/>OpenInference, evals, embeddings drift]
  C --> TP[Tempo/Jaeger<br/>infra traces]
  V -- Prometheus scrape --> P[Prometheus]
  G --> P
  App -- /metrics --> P
  P --> GF[Grafana dashboards + Alertmanager]
  LF --> EV[Online eval workers]
  EV --> LF
  LF --> DW[(Warehouse: ClickHouse/BigQuery<br/>cost · usage · drift)]
```

1.3 OpenTelemetry for GenAI

OTel defines **semantic conventions for GenAI** (gen_ai.* attributes). These conventions are still in development: attribute names (for example gen_ai.system versus gen_ai.provider.name) and how prompt/completion content is captured have changed between releases, so verify against the current specification and pin the semconv version your instrumentation emits. Key span attributes:

Attribute	Example
gen_ai.provider.name (older: gen_ai.system)	vllm, openai
gen_ai.operation.name	chat, embeddings, execute_tool, invoke_agent
gen_ai.request.model / gen_ai.response.model	llama-3.1-8b-instruct
gen_ai.request.temperature, .max_tokens, .top_p	0.2, 512
gen_ai.usage.input_tokens / output_tokens	3120 / 211
gen_ai.response.finish_reasons	["stop"]
gen_ai.response.id	chatcmpl-...

Standard GenAI metrics: gen_ai.client.token.usage (histogram), gen_ai.client.operation.duration, gen_ai.server.time_to_first_token, gen_ai.server.time_per_output_token, gen_ai.server.request.duration. Prompt/completion *content* is opt-in (privacy): record as span events or external references, and redact PII. The redaction rules in section 3.2 must target wherever your instrumentation actually puts content (span attributes such as gen_ai.prompt/gen_ai.completion in older or custom setups, events or external references in newer ones).

OpenInference (Arize) is a complementary convention (span kinds LLM, RETRIEVER, RERANKER, EMBEDDING, TOOL, AGENT, GUARDRAIL, CHAIN, EVALUATOR) with auto-instrumentors for LangChain, LlamaIndex, OpenAI, vLLM-OpenAI clients, etc. Phoenix and LangFuse both ingest OTLP.

1.4 The tools

Tool	Role	Notes
LangFuse (OSS, self-host)	Trace store + UI, prompt management, datasets, evals/scores, cost tracking, user/session views	ClickHouse + Postgres + Redis + S3 backend (v3); OTLP endpoint; Python/JS SDK decorators
Arize Phoenix (OSS)	Notebook/dev-first tracing + evals + **embedding drift/UMAP**, retrieval analysis, experiment comparison	OpenInference native; single container
Prometheus + Grafana	Time-series metrics + alerting	vLLM/TGI/Triton export Prometheus natively
DCGM Exporter	GPU util, memory, temperature, Xid errors, power	Needed for fault detection
OTel Collector	Vendor-neutral pipeline	Batching, attribute redaction, tail-based sampling, fan-out

Tool	Role	Notes
Tempo/Jaeger	General distributed traces	Correlate LLM spans with DB/service spans

1.5 Latency anatomy of one streamed response

```
t0 request arrives
 ■■ network + gateway/auth/guardrails      (Tgw)
 ■■ cache lookup / retrieval / rerank      (Tret)
 ■■ queue wait in inference engine         (Tq)
 ■■ prefill                  (Tpf)    ■■■ first token  =  TTFT = Tgw+Tret+Tq+Tpf
 ■■ decode token 2..N, each ITL_i              ■■■ E2E = TTFT + Σ ITL_i
 ■■ output rails / post-processing
```

Definitions used throughout: - **TTFT** (Time to First Token): request received → first output token delivered. UX metric for "responsiveness". - **ITL / TPOT** (Inter-Token Latency / Time Per Output Token): mean gap between consecutive output tokens after the first. UX metric for "streaming smoothness". Human reading speed ≈ 5–8 tokens/s → ITL ≤ 100–150 ms is acceptable for chat; ≤ 30–50 ms for agents/voice. - **TPS**: per-user = 1/ITL; system = total output tokens / second (throughput). - **E2E latency** = TTFT + (N–1)·ITL (+ post-processing). - **Goodput** = requests/s meeting **all** SLOs.

1.6 What to monitor — the LLMOps golden signals

Category	Signals
Availability / errors	HTTP 5xx, 429, timeouts, engine restarts, guardrail errors, tool failure rate, `finish_reason=length` rate
Latency	TTFT p50/p95/p99, ITL p95/p99, E2E, queue time, retrieval latency, rerank latency
Saturation	num_requests_waiting, num_requests_running, KV-cache usage, preemptions, GPU memory, batch size distribution
Cost & usage	tokens in/out by model/tenant/feature, cache-hit tokens, $ per request/user/feature, GPU-hours, utilisation
Quality	groundedness, answer relevance, citation validity, refusal rate, JSON/tool-call validity, thumbs-down %, escalation rate
Safety	guardrail block rate by category, injection detections, PII detections, canary leaks
Drift	input embedding drift, retrieval score drift, output length/style drift, judge-score drift, model-provider silent changes
Business	task success, deflection, CSAT, conversion

1.7 Hallucination drift detection

Hallucination can't be measured exactly in production (no ground truth); use **proxy signals** and detect **change**:

1. **Judge-based**: sampled groundedness/faithfulness score per trace (Module 07).
2. **Self-consistency**: sample k=3 responses; low agreement (semantic entropy) ⇒ likely hallucination.
3. **Citation checks**: cited doc IDs exist; quoted spans appear in the cited doc (exact/fuzzy match).
4. **Numeric/entity grounding**: numbers/entities in the answer must appear in context.
5. **Retrieval-confidence gating**: low top-1 rerank score + confident answer ⇒ suspicious.
6. **User signals**: thumbs-down, regenerate, copy-rate, "that's wrong" phrases, escalation.
7. **Drift tests** on the above series: CUSUM / EWMA / PSI / KS tests vs. a reference window, segmented by model, prompt, index version, tenant.

---

2. Mathematical Foundations & Formulas

2.1 Latency metrics from timestamps

For request with timestamps: arrival t_a, first token t_1, token i at t_i, last token t_N:

```
TTFT = t_1 – t_a
ITL_i = t_i – t_{i–1},        i = 2..N
ITL_mean (TPOT) = (t_N – t_1) / (N – 1)
E2E = t_N – t_a = TTFT + (N–1)·TPOT
TPS_user = (N–1) / (t_N – t_1) = 1 / TPOT
Throughput_system = Σ_requests N_r / T_window
```

2.2 Percentiles from histograms (Prometheus)

Prometheus histograms store cumulative buckets le. histogram_quantile(q, rate(bucket[5m])) linearly interpolates within the bucket containing the q-th rank: error is bounded by bucket width, so choose buckets around the SLO threshold (e.g., TTFT: 0.05, 0.1, 0.25, 0.5, 0.75, 1, 2, 5 s). **Never average percentiles across instances**; aggregate buckets first (sum by (le)), then compute the quantile.

2.3 SLOs, error budgets and burn rate

SLO target S over window W (e.g., 99 % of requests with TTFT ≤ 700 ms over 28 days). Error budget B = 1 – S. Burn rate:

```
burn = (observed bad fraction over a window) / B
```

Multi-window multi-burn-rate alerting (Google SRE): page when burn ≥ 14.4 over both 1 h and 5 min windows (budget exhausted in ~2 days), ticket when burn ≥ 6 over 6 h & 30 min, or burn ≥ 1 over 3 days.

2.4 Cost attribution

Per request r with model m:

```
cost_r = in_tokens_r·p_in(m) + cached_tokens_r·p_cached(m) + out_tokens_r·p_out(m)
```

Self-hosted: derive effective p from GPU economics:

```
p_out ≈ (GPU_$/h × n_gpu) / (output_tok_per_s × 3600 × utilisation_billing_factor)
```

Allocate shared GPU cost to tenants proportionally to **GPU-seconds** (≈ tokens weighted by prefill_tokens·w_p + decode_tokens·w_d, with w_p ≈ 0.1–0.3 × w_d per token), not request count. Reconcile: Σ_tenants allocated = invoice.

2.5 Drift statistics

Population Stability Index over binned distributions P (reference) and Q (current):

```
PSI = Σ_i (Q_i – P_i) · ln(Q_i / P_i)       < 0.1 stable, 0.1–0.25 moderate, > 0.25 majo
r
```

KS test statistic for continuous scores: D = sup_x |F_P(x) – F_Q(x)|.

Embedding drift: mean cosine distance between the centroid of current-window query embeddings and the reference centroid, or MMD with RBF kernel:

```
MMD^2 = E[k(x,x′)] + E[k(y,y′)] – 2E[k(x,y)]
```

CUSUM for shifts in mean groundedness x_t (reference mean μ■, slack k):

```
S_t = max(0, S_{t–1} + (μ■ – x_t) – k);   alert if S_t > h
```

2.6 Sampling economics

Full-payload tracing cost: traces/s × bytes/trace. With 200 RPS × 25 KB (prompt+completion+spans) = 5 MB/s ≈ 430 GB/day. **Tail-based sampling** keeps 100 % of errors/slow/low-score/flagged traces and p% of healthy ones: expected storage = base·(f_bad + p(1 – f_bad)). Keep metrics at 100 % (cheap), traces sampled.

2.7 Detecting a regression with a Wilson interval

For online binary signals (e.g., thumbs-down): flag if the Wilson upper bound of the current rate is below… or lower bound above the baseline rate. For rare events (0.5 % guardrail blocks) prefer Poisson/exact methods and larger windows.

2.8 Little's Law for concurrency monitoring

L = λW: expected in-flight requests = RPS × mean E2E. If vllm:num_requests_running + waiting deviates strongly from λ·W, latencies or arrivals changed — useful sanity check.

---

3. Production-Ready Executable Code

3.1 Requirements

```
# requirements-observability.txt
opentelemetry-api>=1.26
opentelemetry-sdk>=1.26
opentelemetry-exporter-otlp>=1.26
opentelemetry-instrumentation-fastapi>=0.47b0
opentelemetry-instrumentation-httpx>=0.47b0
openinference-instrumentation-openai>=0.1.12
openinference-instrumentation-langchain>=0.1.28
arize-phoenix>=5.0
langfuse>=2.50
prometheus-client>=0.20
fastapi>=0.112
openai>=1.40
numpy>=1.26
scipy>=1.13
```

3.2 Self-hosted Observability Stack (Docker Compose: LangFuse v3 + OTel Collector + Prometheus + Grafana + Phoenix)

```
# docker-compose.observability.yml
services:
  langfuse-web:
    image: langfuse/langfuse:3
    depends_on: [postgres, clickhouse, redis, minio]
    ports: ["3000:3000"]
    environment: &lf-env
      DATABASE_URL: postgresql://langfuse:langfuse@postgres:5432/langfuse
      NEXTAUTH_URL: http://localhost:3000
      NEXTAUTH_SECRET: ${LANGFUSE_NEXTAUTH_SECRET:?set}
      SALT: ${LANGFUSE_SALT:?set}
      ENCRYPTION_KEY: ${LANGFUSE_ENCRYPTION_KEY:?set 64 hex chars}
      CLICKHOUSE_URL: http://clickhouse:8123
      CLICKHOUSE_MIGRATION_URL: clickhouse://clickhouse:9000
      CLICKHOUSE_USER: clickhouse
      CLICKHOUSE_PASSWORD: clickhouse
      CLICKHOUSE_CLUSTER_ENABLED: "false"
      REDIS_HOST: redis
      REDIS_AUTH: ${REDIS_AUTH:-myredissecret}
      LANGFUSE_S3_EVENT_UPLOAD_BUCKET: langfuse
      LANGFUSE_S3_EVENT_UPLOAD_ENDPOINT: http://minio:9000
      LANGFUSE_S3_EVENT_UPLOAD_ACCESS_KEY_ID: minio
      LANGFUSE_S3_EVENT_UPLOAD_SECRET_ACCESS_KEY: miniosecret
```

```
    LANGFUSE_S3_EVENT_UPLOAD_FORCE_PATH_STYLE: "true"
    TELEMETRY_ENABLED: "false"
  langfuse-worker:
    image: langfuse/langfuse-worker:3
    depends_on: [postgres, clickhouse, redis, minio]
    environment: *lf-env
  postgres:
    image: postgres:16
    environment: { POSTGRES_USER: langfuse, POSTGRES_PASSWORD: langfuse, POSTGRES_DB: langfuse }
    volumes: ["lf-pg:/var/lib/postgresql/data"]
  clickhouse:
    image: clickhouse/clickhouse-server:24.8
    environment: { CLICKHOUSE_USER: clickhouse, CLICKHOUSE_PASSWORD: clickhouse }
    volumes: ["lf-ch:/var/lib/clickhouse"]
  redis:
    image: redis:7
    command: ["redis-server", "--requirepass", "${REDIS_AUTH:-myredissecret}&quo
t;]
  minio:
    image: minio/minio
    entrypoint: sh
    command: -c 'mkdir -p /data/langfuse && minio server /data --address ":9000"'
    environment: { MINIO_ROOT_USER: minio, MINIO_ROOT_PASSWORD: miniosecret }
    volumes: ["lf-minio:/data"]
  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.108.0
    command: ["--config=/etc/otel/config.yaml"]
    volumes: ["./otel-collector.yaml:/etc/otel/config.yaml:ro"]
    ports: ["4317:4317", "4318:4318"]
    environment:
      LANGFUSE_AUTH: ${LANGFUSE_OTLP_BASIC_AUTH:-}      # base64(public:secret)
  phoenix:
    image: arizephoenix/phoenix:latest   # dev convenience only: pin a tested tag in production
    ports: ["6006:6006"]
  prometheus:
    image: prom/prometheus:v2.54.1
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["9090:9090"]
  grafana:
    image: grafana/grafana:11.2.0
    ports: ["3001:3000"]
    volumes: ["./grafana/provisioning:/etc/grafana/provisioning:ro"]
  dcgm-exporter:
    image: nvcr.io/nvidia/k8s/dcgm-exporter:3.3.7-3.5.0-ubuntu22.04
    runtime: nvidia
    cap_add: [SYS_ADMIN]
    ports: ["9400:9400"]
volumes: { lf-pg: {}, lf-ch: {}, lf-minio: {} }
```

otel-collector.yaml — redaction, tail sampling, fan-out:

```
receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }
processors:
  memory_limiter: { check_interval: 1s, limit_mib: 1024 }
  batch: { send_batch_size: 1024, timeout: 5s }
  attributes/redact:
    actions:
      - { key: http.request.header.authorization, action: delete }
      - { key: user.email, action: hash }
  transform/pii:
    trace_statements:
```

```
    - context: span
      statements:
        # mask emails and long digit runs in prompt/completion attributes
        - replace_pattern(attributes["gen_ai.prompt"], "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+
\\.[A-Za-z]{2,}", "<EMAIL>")
        - replace_pattern(attributes["gen_ai.completion"], "[A-Za-z0-9._%+-]+@[A-Za-z0-9
.-]+\\.[A-Za-z]{2,}", "<EMAIL>")
        - replace_pattern(attributes["gen_ai.prompt"], "\\b\\d{13,19}\\b", "&l
t;NUM>")
  tail_sampling:
    decision_wait: 15s
    num_traces: 200000
    policies:
      - { name: errors, type: status_code, status_code: { status_codes: [ERROR] } }
      - { name: slow, type: latency, latency: { threshold_ms: 5000 } }
      - { name: flagged, type: string_attribute, string_attribute: { key: app.flagged, values: ["tru
e"] } }
      - { name: low_quality, type: numeric_attribute, numeric_attribute: { key: eval.groundedness, min_va
lue: 0, max_value: 0.6 } }
      - { name: baseline, type: probabilistic, probabilistic: { sampling_percentage: 10 } }
exporters:
  otlphttp/langfuse:
    endpoint: http://langfuse-web:3000/api/public/otel
    headers: { Authorization: "Basic ${env:LANGFUSE_AUTH}" }
  otlp/phoenix: { endpoint: phoenix:4317, tls: { insecure: true } }
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes/redact, transform/pii, tail_sampling, batch]
      exporters: [otlphttp/langfuse, otlp/phoenix]
```

3.3 Application Instrumentation: OTel GenAI Spans + Streaming TTFT/ITL Metrics

```
"""FastAPI gateway to vLLM with OTel traces and Prometh
eus metrics for TTFT / ITL / TPS / tokens / cost."""
from __future__ import annotations

import json
import time
from typing import AsyncIterator

import httpx
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
from opentelemetry import metrics, trace
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from prometheus_client import Counter, Gauge, Histogram, make_asgi_app

resource = Resource.create({"service.name": "llm-gateway", "service.version"
:: "2026.10.3", "deployment.environment": "prod"})
provider = TracerProvider(resource=resource)
provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter(endpoint="http://otel-collector:4317
", insecure=True)))
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("llm-gateway")

TTFT_BUCKETS = (0.05, 0.1, 0.2, 0.35, 0.5, 0.7, 1.0, 1.5, 2.5, 5.0, 10.0)
ITL_BUCKETS = (0.005, 0.01, 0.02, 0.03, 0.04, 0.06, 0.1, 0.15, 0.25, 0.5)
LABELS = ("model", "tenant", "feature", "route")
```

```
TTFT = Histogram("llm_ttft_seconds", "Time to first token", LABELS, buckets=TTFT_BUCK
ETS)
ITL = Histogram("llm_itl_seconds", "Mean inter-token latency per request", LABELS, bu
ckets=ITL_BUCKETS)
E2E = Histogram("llm_e2e_seconds", "End-to-end latency", LABELS, buckets=(0.25, 0.5,
1, 2, 4, 8, 16, 32, 64))
TOK_IN = Counter("llm_input_tokens_total", "Prompt tokens", LABELS)
TOK_OUT = Counter("llm_output_tokens_total", "Completion tokens", LABELS)
TOK_CACHED = Counter("llm_cached_input_tokens_total", "Prompt tokens served from prefix ca
che", LABELS)
COST = Counter("llm_cost_usd_total", "Attributed cost in USD", LABELS)
REQS = Counter("llm_requests_total", "Requests", LABELS + ("status", "
finish_reason"))
INFLIGHT = Gauge("llm_inflight_requests", "In-flight streaming requests", ("mode
l",))

PRICE = {"llm-primary": {"in": 0.05e-6, "cached": 0.01e-6, "out":
 0.20e-6}}      # derived from GPU economics (§2.4)

app = FastAPI()
FastAPIInstrumentor.instrument_app(app)
HTTPXClientInstrumentor().instrument()
app.mount("/metrics", make_asgi_app())
client = httpx.AsyncClient(base_url="http://vllm:8000/v1", timeout=httpx.Timeout(300.0, connect
=5.0))


@app.post("/v1/chat/completions")
async def chat(request: Request) -> StreamingResponse:
    body = await request.json()
    model = body.get("model", "llm-primary")
    tenant = request.headers.get("x-tenant-id", "unknown")
    feature = request.headers.get("x-feature", "chat")
    route = request.headers.get("x-route", "default")
    labels = dict(model=model, tenant=tenant, feature=feature, route=route)
    body["stream"] = True
    body.setdefault("stream_options", {"include_usage": True})          # usage chunk
 with exact token counts
    return StreamingResponse(_stream(body, labels), media_type="text/event-stream")


async def _stream(body: dict, labels: dict[str, str]) -> AsyncIterator[bytes]:
    t0 = time.perf_counter()
    first: float | None = None
    last = t0
    n_out = 0
    usage: dict = {}
    finish = "unknown"
    status = "ok"
    INFLIGHT.labels(labels["model"]).inc()
    with tracer.start_as_current_span("chat " + labels["model"], kind=trace.SpanKind.
CLIENT) as span:
        span.set_attribute("gen_ai.operation.name", "chat")
        span.set_attribute("gen_ai.provider.name", "vllm")   # older semconv: gen_ai.
system; verify current spec
        span.set_attribute("gen_ai.request.model", labels["model"])
        span.set_attribute("gen_ai.request.max_tokens", body.get("max_tokens", -1))
        span.set_attribute("gen_ai.request.temperature", body.get("temperature", 1.0)
)
        span.set_attribute("app.tenant", labels["tenant"])
        span.set_attribute("app.feature", labels["feature"])
        try:
            async with client.stream("POST", "/chat/completions", json=body) as r:
                if r.status_code != 200:
                    status = f"http_{r.status_code}"
```

```
                span.set_status(trace.Status(trace.StatusCode.ERROR, status))
            async for line in r.aiter_lines():
                if not line.startswith("data: "):
                    continue
                payload = line[6:]
                if payload == "[DONE]":
                    yield b"data: [DONE]\n\n"
                    break
                obj = json.loads(payload)
                if obj.get("usage"):
                    usage = obj["usage"]
                ch = (obj.get("choices") or [{}])[0]
                if ch.get("delta", {}).get("content"):
                    now = time.perf_counter()
                    if first is None:
                        first = now - t0
                        span.add_event("first_token", {"ttft_s": first})
                    n_out += 1
                    last = now
                if ch.get("finish_reason"):
                    finish = ch["finish_reason"]
                yield (line + "\n\n").encode()
    except Exception as e:                                  # client disconnects, upstream r
esets
        status = type(e).__name__
        span.record_exception(e)
        span.set_status(trace.Status(trace.StatusCode.ERROR, status))
        raise
    finally:
        INFLIGHT.labels(labels["model"]).dec()
        e2e = time.perf_counter() - t0
        in_t = usage.get("prompt_tokens", 0)
        out_t = usage.get("completion_tokens", n_out)
        cached = (usage.get("prompt_tokens_details") or {}).get("cached_tokens",
0)
        p = PRICE.get(labels["model"], PRICE["llm-primary"])
        cost = (in_t - cached) * p["in"] + cached * p["cached"] + out_t * p["
;out"]
        if first is not None:
            TTFT.labels(**labels).observe(first)
            if n_out > 1:
                itl = (last - t0 - first) / (n_out - 1)
                ITL.labels(**labels).observe(itl)
                span.set_attribute("llm.itl_mean_s", itl)
                span.set_attribute("llm.tps_user", 1 / itl if itl > 0 else 0)
            span.set_attribute("llm.ttft_s", first)
        E2E.labels(**labels).observe(e2e)
        TOK_IN.labels(**labels).inc(in_t); TOK_OUT.labels(**labels).inc(out_t); TOK_CACHED.labels(**l
abels).inc(cached)
        COST.labels(**labels).inc(cost)
        REQS.labels(**labels, status=status, finish_reason=finish).inc()
        span.set_attribute("gen_ai.usage.input_tokens", in_t)
        span.set_attribute("gen_ai.usage.output_tokens", out_t)
        span.set_attribute("gen_ai.response.finish_reasons", [finish])
        span.set_attribute("llm.cost_usd", cost)
        span.set_attribute("llm.cached_tokens", cached)
```

3.4 LangFuse Python SDK: Nested Traces, Generations, Scores, Cost, Sessions

```
"""LangFuse decorator-based tracing for a RAG pipeline
with user/session/tenant tags and scoring."""
from __future__ import annotations

from langfuse import Langfuse, observe
from langfuse.openai import openai          # drop-in wrapper: auto-traces OpenAI-compatible calls (vLLM
```

```
too)

lf = Langfuse()
client = openai.OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")


@observe(as_type="retriever", name="retrieve")
def retrieve(q: str) -> list[dict]:
    hits = [{"id": "doc_1", "text": "Rotate keys in Settings > Secu
rity.", "score": 0.83}]
    return hits


@observe(as_type="generation", name="answer")
def generate(q: str, hits: list[dict]) -> str:
    ctx = "\n".join(h["text"] for h in hits)
    r = client.chat.completions.create(
        model="llm-primary", temperature=0,
        messages=[{"role": "system", "content": "Answer from context o
nly."}, {"role": "user", "content": f"{ctx}\n\nQ: {q}"}],
        metadata={"prompt_version": 23, "index_version": "kb_v3"},
    )
    return r.choices[0].message.content or ""


@observe(name="rag_request")
def rag(q: str, user_id: str, session_id: str, tenant: str) -> str:
    lf.update_current_trace(user_id=user_id, session_id=session_id, tags=["rag", f"tenant:
{tenant}"],
                            metadata={"tenant": tenant, "release": "2026.10.3&qu
ot;})
    hits = retrieve(q)
    ans = generate(q, hits)
    lf.score_current_trace(name="top1_retrieval_score", value=max(h["score"] for h in
 hits), data_type="NUMERIC")
    return ans


def feedback(trace_id: str, thumbs_up: bool, comment: str = "") -> None:
    lf.create_score(trace_id=trace_id, name="user_feedback", value=1 if thumbs_up else 0, data_
type="BOOLEAN", comment=comment)


if __name__ == "__main__":
    print(rag("How do I rotate an API key?", "u_42", "sess_9", "acme&q
uot;))
    lf.flush()
```

Custom model price definitions for self-hosted models (so LangFuse can compute cost) via API/UI: modelName: llm-primary, matchPattern: (?i)^llm-primary$, inputPrice: 5e-8, outputPrice: 2e-7, unit: TOKENS.

3.5 Arize Phoenix + OpenInference Auto-Instrumentation

```
"""Phoenix: auto-instrument OpenAI-compatible and LangC
hain calls; run evals over traces; view embedding drift."""
from __future__ import annotations

import phoenix as px
from openinference.instrumentation.langchain import LangChainInstrumentor
from openinference.instrumentation.openai import OpenAIInstrumentor
from phoenix.otel import register

tracer_provider = register(project_name="support-rag", endpoint="http://phoenix:6006/v1/tr
aces", batch=True)
OpenAIInstrumentor().instrument(tracer_provider=tracer_provider)
```

```
LangChainInstrumentor().instrument(tracer_provider=tracer_provider)


def run_hallucination_evals() -> None:
    from phoenix.evals import HallucinationEvaluator, OpenAIModel, QAEvaluator, run_evals
    from phoenix.trace import SpanEvaluations
    from phoenix.session.evaluation import get_qa_with_reference

    client = px.Client(endpoint="http://phoenix:6006")
    qa = get_qa_with_reference(client)                          # (input, output, reference-from-ret
rieved-context)
    judge = OpenAIModel(model="llm-judge", base_url="http://judge:8001/v1", api_key=&
quot;EMPTY", temperature=0)
    hallu, qa_eval = run_evals(dataframe=qa, evaluators=[HallucinationEvaluator(judge), QAEvaluator(judge
)], provide_explanation=True)
    client.log_evaluations(SpanEvaluations(eval_name="Hallucination", dataframe=hallu), SpanEva
luations(eval_name="QA Correctness", dataframe=qa_eval))
```

3.6 Prometheus: Scrape Config, Recording Rules, SLO Burn-Rate Alerts

prometheus.yml

```
global: { scrape_interval: 15s, evaluation_interval: 15s }
rule_files: ["rules/*.yml"]
scrape_configs:
  - job_name: vllm
    metrics_path: /metrics
    static_configs: [{ targets: ["vllm:8000"] }]
  - job_name: gateway
    static_configs: [{ targets: ["llm-gateway:8080"] }]
  - job_name: dcgm
    static_configs: [{ targets: ["dcgm-exporter:9400"] }]
```

rules/llm.yml

```
groups:
- name: llm-recording
  interval: 30s
  rules:
  - record: llm:ttft_seconds:p95_5m
    expr: histogram_quantile(0.95, sum by (le, model) (rate(llm_ttft_seconds_bucket[5m])))
  - record: llm:itl_seconds:p95_5m
    expr: histogram_quantile(0.95, sum by (le, model) (rate(llm_itl_seconds_bucket[5m])))
  - record: llm:tokens_per_second:rate1m
    expr: sum by (model) (rate(llm_output_tokens_total[1m]))
  - record: llm:cost_usd:rate1h
    expr: sum by (tenant, feature) (increase(llm_cost_usd_total[1h]))
  - record: llm:ttft_slo_bad_ratio:5m
    expr: 1 - (sum(rate(llm_ttft_seconds_bucket{le="0.7"}[5m])) / sum(rate(llm_ttft_seconds_cou
nt[5m])))
  - record: llm:ttft_slo_bad_ratio:1h
    expr: 1 - (sum(rate(llm_ttft_seconds_bucket{le="0.7"}[1h])) / sum(rate(llm_ttft_seconds_cou
nt[1h])))
  - record: llm:ttft_slo_bad_ratio:6h
    expr: 1 - (sum(rate(llm_ttft_seconds_bucket{le="0.7"}[6h])) / sum(rate(llm_ttft_seconds_cou
nt[6h])))
  - record: llm:ttft_slo_bad_ratio:30m
    expr: 1 - (sum(rate(llm_ttft_seconds_bucket{le="0.7"}[30m])) / sum(rate(llm_ttft_seconds_co
unt[30m])))

- name: llm-alerts
  rules:
  # SLO: 99% of requests TTFT <= 0.7s  =>  error budget 1%
  - alert: TTFTSLOFastBurn
```

```
    expr: llm:ttft_slo_bad_ratio:1h > (14.4 * 0.01) and llm:ttft_slo_bad_ratio:5m > (14.4 * 0.01)
    for: 2m
    labels: { severity: page }
    annotations: { summary: "TTFT SLO budget burning 14.4x (exhausted in ~2 days)" }
  - alert: TTFTSLOSlowBurn
    expr: llm:ttft_slo_bad_ratio:6h > (6 * 0.01) and llm:ttft_slo_bad_ratio:30m > (6 * 0.01)
    for: 15m
    labels: { severity: ticket }
  - alert: LLMErrorRateHigh
    expr: sum(rate(llm_requests_total{status!="ok"}[5m])) / sum(rate(llm_requests_total[5m])) &
gt; 0.02
    for: 5m
    labels: { severity: page }
  - alert: TruncatedOutputsHigh
    expr: sum(rate(llm_requests_total{finish_reason="length"}[15m])) / sum(rate(llm_requests_to
tal[15m])) > 0.03
    for: 15m
    labels: { severity: ticket }
    annotations: { summary: "max_tokens too low or runaway generations" }
  - alert: CostAnomaly
    expr: sum(rate(llm_cost_usd_total[15m])) > 3 * sum(rate(llm_cost_usd_total[15m] offset 1d))
    for: 20m
    labels: { severity: page }
    annotations: { summary: "LLM spend >3x same time yesterday" }
  - alert: TenantCostSpike
    expr: sum by (tenant) (increase(llm_cost_usd_total[1h])) > 5 * avg_over_time(sum by (tenant) (incr
ease(llm_cost_usd_total[1h]))[7d:1h])
    for: 30m
    labels: { severity: ticket }
  - alert: GPUXidError
    expr: increase(DCGM_FI_DEV_XID_ERRORS[5m]) > 0
    labels: { severity: page }
  - alert: PrefixCacheHitRateLow
    expr: sum(rate(llm_cached_input_tokens_total[30m])) / sum(rate(llm_input_tokens_total[30m])) < 0.2
5
    for: 30m
    labels: { severity: ticket }
```

3.7 Online Quality Monitoring Worker: Judge Scores + Drift Detection (PSI / KS / CUSUM)

```
"""Pulls sampled traces, computes proxy hallucination s
ignals, and detects drift vs a reference window."""
from __future__ import annotations

import re
from dataclasses import dataclass
from typing import Sequence

import numpy as np
from scipy.stats import ks_2samp


def psi(ref: Sequence[float], cur: Sequence[float], bins: int = 10) -> float:
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1)); edges[0], edges[-1] = -np.inf, np.inf
    p = np.histogram(ref, edges)[0] / len(ref); q = np.histogram(cur, edges)[0] / len(cur)
    p, q = np.clip(p, 1e-6, None), np.clip(q, 1e-6, None)
    return float(np.sum((q - p) * np.log(q / p)))


@dataclass
class Cusum:
    mu0: float
    k: float = 0.02
```

```
    h: float = 0.5
    s: float = 0.0

    def update(self, x: float) -> bool:
        self.s = max(0.0, self.s + (self.mu0 - x) - self.k)
        return self.s > self.h


NUM = re.compile(r"\b\d[\d,\.]*\b")


def numeric_grounding(answer: str, contexts: list[str]) -> float:
    """Fraction of numbers in the answer that appear verbatim in the context (cheap halluc
ination proxy)."""
    nums = {n.rstrip(".,") for n in NUM.findall(answer)}
    if not nums:
        return 1.0
    ctx = " ".join(contexts)
    return sum(1 for n in nums if n in ctx) / len(nums)


def citation_validity(answer: str, retrieved_ids: set[str]) -> float:
    cited = set(re.findall(r"\[(doc_[a-z0-9]+)\]", answer))
    if not cited:
        return 0.0 if retrieved_ids else 1.0
    return len(cited & retrieved_ids) / len(cited)


def drift_report(ref: dict[str, list[float]], cur: dict[str, list[float]]) -> dict[str, dict[str, floa
t]]:
    out: dict[str, dict[str, float]] = {}
    for metric in ref:
        ks = ks_2samp(ref[metric], cur[metric])
        out[metric] = {"psi": psi(ref[metric], cur[metric]), "ks_stat": float(ks.stat
istic), "ks_p": float(ks.pvalue),
                       "mean_ref": float(np.mean(ref[metric])), "mean_cur": float(np.
mean(cur[metric]))}
    return out


def alert_needed(rep: dict[str, dict[str, float]]) -> list[str]:
    return [m for m, r in rep.items() if r["psi"] > 0.25 or (r["ks_p"] < 0.001
and abs(r["mean_cur"] - r["mean_ref"]) > 0.03)]
```

Embedding-space input drift (query distribution shifts → new topics the system isn't evaluated on):

```
def embedding_drift(ref_vecs: np.ndarray, cur_vecs: np.ndarray, gamma:
 float = 1.0) -> float:
    """Unbiased-ish MMD^2 with an RBF kernel on L2-normalised embeddings, subsampled for s
peed."""
    rng = np.random.default_rng(0)
    a = ref_vecs[rng.choice(len(ref_vecs), min(2000, len(ref_vecs)), replace=False)]
    b = cur_vecs[rng.choice(len(cur_vecs), min(2000, len(cur_vecs)), replace=False)]
    def k(x: np.ndarray, y: np.ndarray) -> np.ndarray:
        d2 = (x**2).sum(1)[:, None] + (y**2).sum(1)[None, :] - 2 * x @ y.T
        return np.exp(-gamma * d2)
    return float(k(a, a).mean() + k(b, b).mean() - 2 * k(a, b).mean())
```

3.8 Cost Attribution Pipeline (per tenant / feature / model) with Reconciliation

```
-- ClickHouse: usage fact table fed from OTel/LangFuse export
CREATE TABLE llm_usage (
  ts DateTime64(3), trace_id String, tenant LowCardinality(String), feature LowCardinality(String),
  model LowCardinality(String), in_tokens UInt32, cached_tokens UInt32, out_tokens UInt32,
```

```
  gpu_seconds Float32, cost_usd Float32, status LowCardinality(String)
) ENGINE = MergeTree PARTITION BY toYYYYMM(ts) ORDER BY (tenant, feature, ts);

-- Daily chargeback, with GPU fleet cost allocated by GPU-seconds
WITH fleet AS (SELECT sum(gpu_hours * hourly_rate) AS fleet_cost FROM gpu_billing WHERE day = today() - 1
),
     use AS (SELECT tenant, feature, sum(gpu_seconds) AS gs FROM llm_usage WHERE toDate(ts) = today() - 1
 GROUP BY tenant, feature)
SELECT tenant, feature, gs,
       gs / sum(gs) OVER () * (SELECT fleet_cost FROM fleet) AS allocated_cost_usd
FROM use ORDER BY allocated_cost_usd DESC;
```

```
def estimate_gpu_seconds(prefill_tokens: int, decode_tokens: int, batc
h_avg: float,
                         prefill_tok_per_s: float = 45_000, decode_step_s: float = 0.012) -> float:
    """Approximate GPU time consumed by a request: prefill is compute-exclusive; decode ti
me is shared across the batch."""
    return prefill_tokens / prefill_tok_per_s + decode_tokens * decode_step_s / max(1.0, batch_avg)
```

3.9 Grafana Dashboard Panels (PromQL cheat-sheet)

```
# TTFT p50/p95/p99 by model
histogram_quantile(0.95, sum by (le, model) (rate(llm_ttft_seconds_bucket[5m])))

# Inter-token latency p95
histogram_quantile(0.95, sum by (le, model) (rate(llm_itl_seconds_bucket[5m])))

# System output tokens/sec and per-user tokens/sec (median)
sum(rate(llm_output_tokens_total[1m]))
1 / histogram_quantile(0.5, sum by (le) (rate(llm_itl_seconds_bucket[5m])))

# vLLM saturation
avg(vllm:gpu_cache_usage_perc)                  # KV cache fill
sum(vllm:num_requests_waiting)                  # queue
rate(vllm:num_preemptions_total[5m])            # preemption churn

# Cost per 1K requests by feature
1000 * sum by (feature) (rate(llm_cost_usd_total[1h])) / sum by (feature) (rate(llm_requests_total[1h]))

# Cache efficiency
sum(rate(llm_cached_input_tokens_total[15m])) / sum(rate(llm_input_tokens_total[15m]))

# Truncation rate (finish_reason=length)
sum(rate(llm_requests_total{finish_reason="length"}[15m])) / sum(rate(llm_requests_total[15m]))

# GPU health
avg by (gpu) (DCGM_FI_DEV_GPU_UTIL); max by (gpu) (DCGM_FI_DEV_GPU_TEMP); increase(DCGM_FI_DEV_XID_ERRORS
[1h])
```

3.10 Kubernetes: ServiceMonitor, PrometheusRule, and OTel Auto-config

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: vllm, labels: { release: kube-prometheus-stack } }
spec:
  selector: { matchLabels: { app: vllm } }
  endpoints: [{ port: http, path: /metrics, interval: 15s }]
---
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata: { name: llm-otel }
spec:
```

```
  mode: deployment
  replicas: 2
  config:
    receivers: { otlp: { protocols: { grpc: {}, http: {} } } }
    processors: { batch: {}, memory_limiter: { check_interval: 1s, limit_percentage: 75 } }
    exporters: { otlphttp/langfuse: { endpoint: http://langfuse-web.llmops:3000/api/public/otel, headers:
 { Authorization: "Basic ${env:LANGFUSE_AUTH}" } } }
    service: { pipelines: { traces: { receivers: [otlp], processors: [memory_limiter, batch], exporters:
[otlphttp/langfuse] } } }
```

3.11 Synthetic Canary Probe (black-box SLI)

```
"""Every 30s send a fixed prompt through the full produ
ction path; export success, TTFT and a golden-answer check."""
from __future__ import annotations

import time

import httpx
from prometheus_client import Gauge, start_http_server

OK = Gauge("canary_success", "1 if golden check passed", ["probe"])
TT = Gauge("canary_ttft_seconds", "TTFT of probe", ["probe"])
PROBES = {"faq_rotate_key": ("How do I rotate my API key?", "Settings"), &q
uot;json_tool": ("Return JSON {\"ok\":true}", "\"ok\"")}


def run_once() -> None:
    for name, (prompt, must) in PROBES.items():
        t0 = time.perf_counter(); first = None; text = ""
        try:
            with httpx.stream("POST", "https://llm.internal/v1/chat/completions", tim
eout=30,
                              json={"model": "llm-primary", "stream": True,
 "temperature": 0, "max_tokens": 128,
                                    "messages": [{"role": "user", "con
tent": prompt}]}) as r:
                for line in r.iter_lines():
                    if line.startswith("data: {"):
                        if first is None: first = time.perf_counter() - t0
                        text += line
            OK.labels(name).set(1 if must in text else 0); TT.labels(name).set(first or 30)
        except Exception:
            OK.labels(name).set(0); TT.labels(name).set(30)


if __name__ == "__main__":
    start_http_server(9105)
    while True:
        run_once(); time.sleep(30)
```

---

4. Hard-Learned Production Failures & Edge Cases

Incident 1 — "Average latency is fine" while users suffer (percentile math done wrong)

Symptom. The dashboard showed mean E2E 1.4 s and p95 1.9 s; support tickets complained of 20-second hangs.

Root-cause analysis. 1. The dashboard averaged **per-pod p95 values** across 12 pods; one pod was stuck in preemption thrash with p95 = 40 s but weighted 1/12. 2. TTFT was measured at the engine, not at the user edge; gateway queueing (connection pool exhaustion) added ~15 s invisible to engine metrics. 3. No per-tenant breakdown; the affected tenant sent 30k-token prompts.

Remediation. Aggregate **buckets** then compute quantile (sum by (le)), measure TTFT at the gateway (user-perceived), add tenant and prompt-length bucket labels (bounded cardinality), alert on p99 and on **max per-pod** KV-cache usage and preemptions, add a black-box canary (§3.11), and publish SLOs on user-edge metrics.

Incident 2 — Silent quality regression after provider-side model update

Symptom. Groundedness fell from 0.91 → 0.82 over a week with no deploy on our side; thumbs-down rose 40 %.

Root cause. The hosted-model alias (gpt-4o-style floating alias) was repointed to a new snapshot. No pinning, no version tag in traces, and drift alerts didn't exist.

Remediation. Pin dated model snapshots; record gen_ai.response.model (the *actual* model returned) as a span attribute and metric label; run the canary golden set hourly against the endpoint; CUSUM/PSI alerts on online judge scores (§3.7); contract with providers for deprecation notices; maintain a fallback self-hosted model and a switch.

Incident 3 — PII landed in the observability store (compliance incident)

Symptom. An audit found raw SSNs and medical notes in LangFuse and in Elasticsearch logs.

Root cause. Instrumentation captured full prompts and completions by default; no redaction in the pipeline; retention was "forever"; LangFuse access was broad.

Remediation. Redact at the **collector** (transform processor / Presidio sidecar) before export, not in each app; content capture off by default with per-tenant opt-in for debugging; field-level hashing for identifiers; RBAC and project separation per tenant; retention (e.g., 30 days for payloads, 13 months for aggregates); encryption at rest; DSAR/erasure tooling keyed by user_id; regular scans for PII patterns in the trace store; separate tracing path for regulated tenants (metadata only).

Incident 4 — Metric cardinality explosion took down Prometheus

Symptom. Prometheus OOM-killed; dashboards blank during an incident.

Root cause. Someone added user_id and conversation_id as metric labels to attribute cost; 2M active label combinations.

Remediation. Labels limited to bounded sets (model, tenant, feature, route, status); per-user/per-conversation attribution goes to **traces/warehouse**, not Prometheus; relabel rules drop disallowed labels; cardinality alert (prometheus_tsdb_head_series); use exemplars to link a histogram bucket to a trace_id.

Incident 5 — Streaming cutoff hidden: finish_reason="length" and client disconnects

Symptom. Users reported "answers cut off mid-sentence" yet error rate was 0.

Root cause. Responses ended with HTTP 200 but finish_reason=length (max_tokens=256 vs long answers after a prompt change) and some clients disconnected mid-stream (proxy idle timeout 60 s) – both counted as success.

Remediation. Track finish_reason distribution as a first-class metric (§3.6 TruncatedOutputsHigh); alert on cancel/disconnect ratio; align proxy timeouts (proxy_read_timeout) with generation length and send SSE keep-alive comments; log abort reasons; set max_tokens from p99 output length × 1.3.

Incident 6 — Cost spike from an agent loop only visible a day later

Symptom. Monthly bill +38 %; finance asked "which feature?" — no one could say.

Root cause. Cost was tracked only at the provider invoice level; no per-feature attribution; agent retries inflated tokens 12× for one workflow.

Remediation. Token and cost counters labelled by tenant/feature at the gateway; hourly CostAnomaly and per-tenant spike alerts (§3.6); per-run budget enforcement (Module 09); weekly cost review dashboard with unit economics ($ per resolved ticket); budgets with automatic degradation (route to a smaller model) at thresholds.

Incident 7 — Trace context lost across the async boundary

Symptom. LangFuse showed orphaned LLM spans without parents; couldn't link a bad answer to its retrieval step.

Root cause. Work was handed to a background thread pool/Celery queue without propagating the OTel context; the vLLM client created a new root span.

Remediation. Use opentelemetry.context attach/detach or propagate.inject/extract through message headers (W3C traceparent), instrument Celery/Kafka clients, and add a CI test asserting that a synthetic request yields a single connected trace with expected span names.

---

5. Senior/Staff LLMOps Interview Questions

Scenario 1 — "Define TTFT, ITL and TPS. Which do you put in the SLO and why?"

Junior/Mid Trap Answer: "Latency is how long the request takes; we track average latency and tokens per second."

Staff Production Answer: "They capture different user experiences. TTFT is the wait before anything appears (queue + retrieval + prefill) and dominates perceived responsiveness; ITL/TPOT is streaming smoothness during decode; per-user TPS is 1/ITL; system TPS is aggregate throughput, a capacity/cost metric rather than a UX metric; E2E = TTFT + (N–1)·ITL matters for non-streaming consumers like agents and batch tools. I'd set SLOs on user-edge TTFT p95/p99 and ITL p95 per product tier (chat: TTFT ≤ 700 ms, ITL ≤ 60 ms; agent tool calls care about E2E), express them as burn-rate-alerted error budgets, and track *goodput* — requests meeting all SLOs — instead of average latency. I'd bucket by prompt length because TTFT scales with prefill, and I'd measure at the gateway so queueing outside the engine is included."

Scenario 2 — "How would you instrument a multi-step RAG agent so a bad answer can be debugged in minutes?"

Junior/Mid Trap Answer: "Log the prompt and the response."

Staff Production Answer: "One trace per user request with a span tree: gateway → guardrails (per rail) → cache → query rewrite → retrieval (with filters, k, scores) → rerank → prompt assembly (prompt name+version, token counts, dropped segments) → each LLM call (gen_ai.* attributes: model actually served, params, tokens, finish reason, cached tokens) → each tool call (args hash, latency, result size, error) → output rails. Attach user_id, session_id, tenant, release, prompt_version, index_version, and the online eval scores to the trace. Propagate W3C trace context across async boundaries and into vLLM so engine-side queue/prefill spans join the trace. Use OpenTelemetry/OpenInference conventions so I can export to LangFuse and Phoenix. Payloads are redacted at the collector and tail-sampled so errors, slow, flagged and low-score traces are kept at 100 %. With that, I can open the trace for a thumbs-down and see immediately whether retrieval missed (context relevance), the model ignored context (groundedness), a rail blocked something, or truncation occurred."

Scenario 3 — "Design hallucination drift detection for a production RAG system with no ground truth."

Junior/Mid Trap Answer: "Ask users to flag hallucinations and count them."

Staff Production Answer: "User flags are sparse and biased, so I build layered proxies: (1) sample 2–5 % of traces for an LLM-judge groundedness score using a calibrated cross-family judge; (2) cheap deterministic checks on 100 % of traffic — citation validity, numeric/entity grounding versus context, answer-without-retrieval rate, retrieval-confidence vs answer-confidence mismatch; (3) self-consistency or semantic-entropy sampling on a subset; (4) behavioural signals (regenerate, copy, thumbs, escalation). I store each as a time series segmented by model, prompt version, index version and tenant. Detection: PSI/KS against a reference window plus CUSUM/EWMA on the mean with a slack parameter, requiring practical significance (e.g., ≥ 3 points) as well as statistical. Alerts open a review task with the offending traces; a weekly human audit of ~100 traces re-calibrates the judge. I also track input drift (embedding MMD on queries) because new topics can degrade quality without any system change, and run a golden-set canary hourly to separate system-side from input-side drift."

Scenario 4 — "Prometheus shows p95 TTFT 400 ms but users say it's slow. What's wrong?"

Junior/Mid Trap Answer: "Users are probably on bad networks."

Staff Production Answer: "Check *where* and *what* is measured. (1) Engine-side TTFT excludes gateway queueing, guardrails, retrieval, rerank and the network, so measure at the user edge and break down TTFT by stage via traces. (2) Averaging per-instance quantiles hides a bad pod; aggregate histogram buckets. (3) Bucket resolution: if the nearest buckets are 0.25 and 1.0, interpolation can hide a shift around the SLO; add buckets near the threshold. (4) Tenant or prompt-length skew: the p95

across all traffic hides a heavy-tenant p95 of 6 s; slice by tenant and prompt-length. (5) Coordinated omission: if the load generator or client times out, slow requests vanish from the metric; count timeouts as failures. (6) Time-to-first-*visible*-token — some UIs buffer or the first token is whitespace/reasoning tokens. I'd validate with a synthetic canary from outside the network and RUM data from the client."

Scenario 5 — "How do you attribute GPU cost to tenants/features fairly?"

Junior/Mid Trap Answer: "Count requests per tenant and divide the bill."

Staff Production Answer: "Requests vary by orders of magnitude in cost, so I allocate by consumed resource. For an API provider it's tokens × price with cached-token discounts. For self-hosted GPUs, I estimate GPU-seconds per request: prefill tokens ÷ prefill throughput (compute-exclusive) plus decode tokens × step time ÷ average batch size (shared), then allocate the actual fleet cost (GPU-hours × rate, including idle) proportionally to GPU-seconds, with a separate line for idle/headroom capacity so unit economics are visible. Data path: gateway emits tokens, cached tokens, model and tenant/feature labels to Prometheus for real-time and to ClickHouse via traces for exact daily reconciliation against the cloud invoice. Per-user detail stays in the warehouse to avoid cardinality blowups. I'd add budgets and alerts per tenant, plus anomaly detection on spend per hour, and report $/resolved-task rather than just $/token."

Scenario 6 — "What do you alert on for an LLM serving cluster (pages vs tickets)?"

Junior/Mid Trap Answer: "Alert on GPU utilisation above 90 % and error rate."

Staff Production Answer: "Page on user-impacting, actionable symptoms: multi-window burn-rate on TTFT/ITL/availability SLOs; 5xx/timeout rate; GPU Xid errors or a hung rank (canary failing); KV-cache usage sustained > 92 % with rising preemptions or queue; cost anomaly; guardrail/injection-detection spike or canary-token leak. Tickets for slower-moving issues: prefix-cache hit-rate decline, truncation (finish_reason=length) rate, quality drift (judge score CUSUM), embedding drift, tool-call validation errors, cold-start time regression, model-load failures. GPU utilisation is *not* a good page — 100 % may be healthy; I'd rather alert on saturation (queue, KV) and symptoms. Each alert has a runbook with likely causes (e.g., KV pressure → check long-context traffic, max-num-seqs, scale-out) and links to dashboards and a trace query."

Scenario 7 — "Is it safe to log prompts and completions? How do you do it responsibly?"

Junior/Mid Trap Answer: "Yes, you need them for debugging — log everything."

Staff Production Answer: "Prompts contain user data, PII, secrets and proprietary documents, so logging is a data-processing activity subject to privacy law and customer contracts. Policy: capture content only where needed, at the lowest fidelity that supports debugging; redact at the collector (PII patterns + NER, secret scanners) before data reaches storage; hash identifiers; per-tenant controls with metadata-only mode for regulated customers; short retention for payloads with legal-hold override; RBAC and audit of who views traces; encryption; region pinning; and erasure workflows by user ID. Keep aggregates and evals (scores, token counts) longer than raw content. Use sampling to reduce exposure. Also treat the observability stack as an attack surface: stored prompts can contain injections that attack reviewers' tools or the eval judges, so render them safely."

Scenario 8 — "Cardinality: how do you track per-user usage without blowing up Prometheus?"

Junior/Mid Trap Answer: "Add a user_id label to the token counter."

Staff Production Answer: "High-cardinality identifiers don't belong on metrics. Prometheus keeps a time series per label combination, so 1M users × models × features is unmanageable. Keep metrics bounded (tenant tier, feature, model, status) for real-time alerts and put per-user/per-conversation data in traces and an analytical store such as ClickHouse or BigQuery, where it's queryable by user ID. For top-N views (heavy users) use a streaming aggregation (Redis sorted sets, count-min sketch) or a periodic top-k query from the warehouse, and use histogram exemplars to jump from a metric spike to representative trace_ids. Enforce label allow-lists in the OTel collector or metric relabeling and alert on prometheus_tsdb_head_series growth."

Key Takeaways

- Instrument **traces, metrics, logs and evals**; use OTel GenAI + OpenInference conventions; export to LangFuse/Phoenix via a Collector that redacts and tail-samples.
- Define **TTFT, ITL, E2E, goodput** at the user edge; alert on **burn rates**, not raw thresholds; aggregate histogram buckets, never average percentiles.

- Attribute cost by **GPU-seconds/tokens** with bounded metric labels; keep per-user detail in the warehouse.
- Detect **quality drift** with layered proxies (judge, deterministic checks, user signals) and statistical drift tests; pin model versions and record the actual served model.
- Observability data is sensitive: **redact early, retain briefly, control access.**

10.6 GenAI Telemetry Design: Semantic Conventions, Content Capture and Sampling Policy

Conceptual Deep Dive

OpenTelemetry's GenAI **semantic conventions** standardise span and metric attributes for model calls (operation name, requested and response model, token usage, finish reasons). They are still evolving (status "development" at the time of writing; verify against the current spec). Names have moved: early drafts used gen_ai.system for the provider and carried prompt and completion text as span attributes (gen_ai.prompt, gen_ai.completion, which the legacy chapter's examples use); later revisions renamed the provider attribute and moved message content into events or opt-in attributes. Design your telemetry layer so attribute names are defined in one mapping module and can be switched with a config flag, and keep a dual-emit period when migrating dashboards.

Span hierarchy that reconstructs an agent run: request, guardrail, retrieval, agent step, LLM call, tool call. Put retrieval results (document ids and scores, not text) and the prompt/bundle id on spans so a bad answer can be traced to the exact configuration. **Content capture** is opt-in: always record metadata (token counts, model, finish reason, latency, TTFT), and record prompt or completion text only under an explicit policy, redacted at the collector so applications cannot bypass it (8.7).

Tail sampling decides after the trace completes: keep every error, slow, expensive and low-judge-score trace plus a small random baseline chosen by hashing the trace id so all services agree. Head sampling at a fixed rate throws away the interesting traces before you know they are interesting.

Cardinality discipline. Metric labels should be bounded sets: model, route, tenant tier, outcome. User ids and prompt hashes go on spans, not on Prometheus labels. Use **exemplars** to attach a trace id to a histogram bucket so an engineer can jump from a latency spike to a concrete trace. Link feedback and eval items to trace ids so production failures become eval cases (14.4).

Edge cases. Async boundaries (queues, background workers, tool callbacks) drop context unless the trace context is propagated explicitly in message headers. Streaming responses: record TTFT as an event on the span; close the span when the stream ends or the client disconnects, otherwise spans leak or report impossible durations.

Cross-Connection Scenarios

- Privacy-safe content capture and redaction (8.7).
- Quality SLIs computed from sampled traces (10.7).
- Trace-id linkage to evaluation datasets (14.4).
- Per-tenant cost attribution from token metrics (11.8).

Production Anti-Patterns

- Sampling at 1% head-based and then wondering where the incident traces are.
- Storing full prompts in span attributes by default: every trace backend user can read customer data and storage cost balloons.
- Per-user labels on Prometheus metrics: series count explodes and the monitoring system degrades or bills unexpectedly.

Minimal Working Code

A tail-sampling decision function (error, latency, cost, judge score, baseline probability by trace-id hash) and a simulation reporting the kept fraction and the recall of bad traces versus head sampling.

```
import hashlib, random

def baseline_hit(trace_id, p):
    h = int(hashlib.sha256(trace_id.encode()).hexdigest()[:8], 16) / 2**32
    return h < p

def keep(t, slow_ms=4000, cost_cap=0.05, low_score=0.5, p=0.02):
    if t["error"] or t["latency_ms"] > slow_ms or t["cost"] > cost_ca
p:
```

```
        return True
    if t["judge"] is not None and t["judge"] < low_score:
        return True
    return baseline_hit(t["id"], p)

random.seed(0)
traces = []
for i in range(20000):
    bad = random.random() < 0.05
    judged = random.random() < 0.3                    # judge sees only some traffic
    traces.append({"id": f"t{i}", "bad": bad,
                   "error": bad and random.random() < 0.3,
                   "latency_ms": random.gauss(1500, 400) + (3000 if bad and random.random() <
; 0.2 else 0),
                   "cost": 0.01 if not bad else random.choice([0.01, 0.2]),
                   "judge": (0.2 if bad else 0.9) if judged else None})

kept = [t for t in traces if keep(t)]
bad_total = sum(t["bad"] for t in traces)
recall = sum(t["bad"] for t in kept) / bad_total
head = [t for t in traces if baseline_hit(t["id"], len(kept) / len(traces))]
head_recall = sum(t["bad"] for t in head) / bad_total
print(f"kept {len(kept)/len(traces):.3f} tail_recall {recall:.3f} head_recall {head_recall:.3f}"
;)
assert recall > head_recall + 0.2
```

A span-attribute sanitiser: allow-list keys, truncate long values, hash high-cardinality fields, and drop content unless a capture flag is set.

```
import hashlib

ALLOW = {"gen_ai.request.model", "gen_ai.usage.input_tokens", "gen_ai.usage.outp
ut_tokens",
         "route", "tenant_tier"}
HASH = {"user.id", "session.id"}
CONTENT = {"prompt", "completion"}

def sanitise(attrs, capture_content=False, max_len=200):
    out = {}
    for k, v in attrs.items():
        if k in ALLOW:
            out[k] = v
        elif k in HASH:
            out[k] = hashlib.sha256(str(v).encode()).hexdigest()[:12]
        elif k in CONTENT and capture_content:
            out[k] = str(v)[:max_len]
        # everything else is dropped
    return out

raw = {"gen_ai.request.model": "MODEL_ID", "gen_ai.usage.input_tokens": 120
, "user.id": "alice@example.com",
       "prompt": "my card is 4111 ...", "completion": "x" * 500,
"debug.secret": "token"}
safe = sanitise(raw)
assert "prompt" not in safe and "debug.secret" not in safe
assert len(safe["user.id"]) == 12 and safe["user.id"] != "alice@example.com"
;
with_content = sanitise(raw, capture_content=True)
assert len(with_content["completion"]) == 200
print(safe)
```

Interview Angle

Q: Design tracing for a multi-step agent so an engineer can reconstruct a bad run without exposing PII. **A:** One trace per request with child spans per agent step, LLM call and tool call; spans carry ids, scores, token counts, tool names and argument shapes,

with content captured only under a policy flag and redacted at the collector. Trade-off: less content means slower debugging, so enable sampled capture for flagged traces with short retention. Check: engineers resolve a seeded incident from traces alone, and a PII scan of stored spans finds none.

Q: How do you keep trace storage affordable while retaining the incident traces? **A:** Tail sampling: keep all errors, slow, expensive and low-score traces plus a hash-based baseline, and drop or shorten retention for the rest. Cost: a buffering collector tier that holds traces until completion. Check: recall of known-bad traces (from user reports) in the kept set stays near 100% and the kept fraction stays within storage budget.

Q: What do you do when the semantic conventions change under you? **A:** Isolate attribute names in one mapping layer, dual-emit old and new names during migration, pin the instrumentation version, and update dashboards and alerts from the mapping. Trade-off: temporary duplicate cost. Check: a contract test asserting emitted attributes match the chosen spec version, and no dashboard panel going empty after cutover.

10.7 Quality SLOs and Alerting for Non-Deterministic Systems

Conceptual Deep Dive

Latency and error SLOs are necessary but do not tell you whether answers are good. Define **quality SLIs** as proportions measured on sampled traffic: judged-correct rate, groundedness, format validity, refusal rate, task success, explicit feedback rate, and success of synthetic canary probes with known answers. Each is a proportion, so each fits the standard SLO machinery: a target (for example 95% judged-correct over 28 days), a windowed measurement and an **error budget** equal to the allowed bad fraction.

Judge-based SLIs are noisy and sampled, so alert logic must be confidence-aware. Use Wilson bounds on the bad proportion and a **minimum sample** size before any alert can fire; an alert on 7 of 10 sampled responses failing is not evidence. Multi-window burn-rate alerts (a fast window and a slow window both over threshold) reduce flapping while catching real regressions. Burn rate is the observed bad rate divided by the budgeted bad rate.

Change detection complements thresholds. After a deploy, compare the new bundle with the previous one on the same segments. For slow drift, run CUSUM or Page-Hinkley on judge scores: they accumulate small persistent deviations and detect shifts that no single window crosses. Compare within segments (language, route, tenant tier) to avoid Simpson effects, where a traffic mix change moves the aggregate without any per-segment change.

Alert hygiene. Page for user-visible breaches with fast burn; open tickets for slow burns; every alert links a runbook (check release markers, bundle diff, provider status, judge health); deduplicate and group by cause. **Provider incident correlation:** canary probes against a pinned prompt and the provider status feed help separate your regression from the provider's. Dashboards need release markers, the bundle id, and per-segment quality panels.

Edge cases. Low-traffic tenants never reach the minimum sample, so aggregate them into cohorts or use long windows. A judge outage produces missing data, which looks like "no bad answers"; alert on missing data (sample count below expected) as a separate condition.

Cross-Connection Scenarios

- Telemetry and trace sampling feeding SLIs (10.6).
- Judge calibration and drift (14.2).
- Release gates and rollback triggers (Module 03, 14.4).
- Incident response and post-mortems (Module 12).

Production Anti-Patterns

- Paging on a noisy 5-minute judge average: operators learn to ignore the pager.
- No minimum sample size: small-n fluctuations create false pages and false confidence.
- Quality alerts without release markers: responders cannot tell whether a deploy caused the drop.

Minimal Working Code

A burn-rate calculator over multiple windows using Wilson bounds and a minimum-sample guard.

```
import math
```

```
def wilson_bounds(bad, n, z=1.96):
    p = bad / n
    d = 1 + z * z / n
    c = (p + z * z / (2 * n)) / d
    h = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / d
    return max(0.0, c - h), min(1.0, c + h)

def burn_alert(windows, slo=0.95, threshold=2.0, min_n=100):
    """windows: list of (bad, n). Fire only if every window has enough samples and its low
er bound burns fast."""
    budget = 1 - slo
    rates = []
    for bad, n in windows:
        if n < min_n:
            return {"fire": False, "reason": "insufficient_samples"}
        lo, _ = wilson_bounds(bad, n)
        rates.append(lo / budget)
    return {"fire": all(r > threshold for r in rates), "burn": [round(r, 2) for r
in rates]}

print(burn_alert([(30, 200), (150, 1500)]))             # sustained
print(burn_alert([(8, 10), (150, 1500)]))               # tiny fast window
assert burn_alert([(30, 200), (400, 1500)])["fire"]
assert burn_alert([(8, 10), (400, 1500)])["reason"] == "insufficient_samples"
assert not burn_alert([(10, 200), (75, 1500)])["fire"]
```

A CUSUM detector on a simulated stream of judge scores with a mean shift, reporting detection delay versus false-alarm rate for several thresholds.

```
import numpy as np

def cusum_alarm(x, mu0, k, h):
    s = 0.0
    for i, v in enumerate(x):
        s = max(0.0, s + (mu0 - v - k))        # one-sided: detects a drop
        if s > h:
            return i
    return None

rng = np.random.default_rng(0)
sigma, change, T, runs = 0.05, 150, 300, 300
res = {}
for h in (0.1, 0.2, 0.4):
    false_alarms, delays = 0, []
    for _ in range(runs):
        x = np.concatenate([rng.normal(0.85, sigma, change), rng.normal(0.80, sigma, T - change)])
        i = cusum_alarm(x, mu0=0.85, k=0.5 * sigma, h=h)
        if i is None:
            continue
        if i < change:
            false_alarms += 1
        else:
            delays.append(i - change)
    res[h] = (false_alarms / runs, float(np.mean(delays)) if delays else float("nan"))
    print(f"h={h}: false_alarm_rate={res[h][0]:.2f} mean_delay={res[h][1]:.1f}")
assert res[0.1][0] >= res[0.4][0]
assert res[0.4][1] >= res[0.1][1]
```

Interview Angle

Q: How do you define and alert on "answer quality" for a production assistant? **A:** Define SLIs as proportions: judged-correct on a calibrated sample, groundedness, format validity, and canary-probe success, with a target and error budget. Alert on multi-window burn rate with Wilson bounds and a minimum sample; page only on user-visible fast burn. Trade-off: judge cost and noise versus coverage. Check: replay historical incidents and confirm detection within the target time with an acceptable false-page count.

Q: Quality dropped silently for two days. What monitoring was missing? **A:** Likely candidates: no sampled quality SLI, alerts on infrastructure metrics only, judge outage treated as no data, no drift detector (CUSUM) for slow decline, no release markers, and no per-segment panels. I would add each with a runbook. Check: inject a synthetic degradation in staging and measure time-to-detect; require it under the agreed bound.

Q: How do you avoid alert fatigue from LLM-judge metrics? **A:** Use confidence bounds and minimum samples, burn-rate windows rather than raw averages, page only on user-visible breaches, route slow burns to tickets, group duplicate alerts and prune alerts nobody acts on. Trade-off: slower detection of small regressions. Check: pages per week, fraction of pages that led to action, and median time-to-acknowledge.